Tx Table (Transaction Table)

목적

Tx Table은 블록 내 **모든 트랜잭션의 필드를 저장**하는 테이블입니다. 이 테이블은 Tx Circuit에서 생성되며, EVM Circuit에서 트랜잭션 정보를 조회하는 데 사용됩니다. 트랜잭션 타입(Legacy, EIP-155, EIP-2930, EIP-1559, L1 Message)에 따라 다양한 필드를 지원합니다.

테이블 스키마

파일: `zkevm-circuits/src/table.rs` (line 212)

```
pub struct TxTable {
/// 행 활성화 여부
pub q_enable: Column<Fixed>,
/// 트랜잭션 ID
pub tx_id: Column<Advice>,
/// 필드 태그 (TxFieldTag)
pub tag: Column<Advice>,
/// 인덱스 (CallData, AccessList용)
pub index: Column<Advice>,
/// 필드 값
pub value: Column<Advice>,
/// Access List 주소
```

```
pub  access_list_address: Column<Advice>,

/// 청크 트랜잭션 바이트 해시 RLC

pub  chunk_txbytes_hash_rlc: Column<Advice>,

}
```

컬럼 설명

1. q_enable (Fixed)

- 해당 행이 유효한 트랜잭션 행인지 표시
- 모든 트랜잭션 행에서 `1`

2. tx_id (Advice)

- 트랜잭션 ID (1부터 시작)
- 블록 내에서 트랜잭션을 고유하게 식별

3. tag (Advice)

- 트랜잭션 필드를 식별하는 태그 (`TxFieldTag`)
- 각 태그에 따라 `value` 컬럼의 의미가 달라짐

4. index (Advice)

- **CallData**: 바이트 인덱스 (0부터 시작)
- **AccessList**: 항목 인덱스
- **다른 필드**: 사용되지 않음 (0)

5. value (Advice, SecondPhase)

- 필드의 실제 값

- RLC가 필요한 경우 SecondPhase에 할당

6. access_list_address (Advice)

- **AccessListAddress**: 주소 값
- **AccessListStorageKey**: 주소 값 (storage key는 value에 저장)
- **다른 필드**: 사용되지 않음 (0)

7. chunk_txbytes_hash_rlc (Advice, SecondPhase)

- 청크 내 모든 L2 트랜잭션 바이트의 Keccak 해시 RLC
- 멀티 블록 증명에서 사용
- 고정 섹션의 마지막 행에만 할당

TxFieldTag 열거형

파일: `zkevm-circuits/src/table.rs` (line 127)

```
pub  enum TxFieldTag {
Null = 0, // 사용되지 않음
CallData, // 트랜잭션 데이터
Nonce, // 발신자 nonce
GasPrice, // 가스 가격 (Legacy, EIP-155)
Gas, // 가스 한도
CallerAddress, // 발신자 주소
CalleeAddress, // 수신자 주소
IsCreate, // 컨트랙트 생성 여부
Value, // 전송 값 (wei)
```

```
CallDataRLC, // CallData RLC

CallDataLength, // CallData 길이

CallDataGasCost, // CallData 가스 비용

TxDataGasCost, // L1 데이터 가스 비용

ChainID, // 체인 ID

SigV, // 서명 V

SigR, // 서명 R

SigS, // 서명 S

TxSignLength, // 서명용 RLP 길이

TxSignRLC, // 서명용 RLP RLC

TxSignHash, // 서명용 해시

TxHashLength, // 서명된 Tx RLP 길이

TxHashRLC, // 서명된 Tx RLP RLC

TxHash, // 트랜잭션 해시

TxType, // 트랜잭션 타입

AccessListAddress, // Access List 주소

AccessListStorageKey, // Access List 스토리지 키

AccessListAddressesLen, // Access List 주소 개수

AccessListStorageKeysLen, // Access List 스토리지 키 개수

AccessListRLC, // Access List RLC

BlockNumber, // 블록 번호

MaxPriorityFeePerGas, // 최대 우선순위 수수료 (EIP-1559)

MaxFeePerGas, // 최대 수수료 (EIP-1559)
```

```
}
```

필드 그룹

기본 트랜잭션 필드

- `Nonce` : 발신자의 트랜잭션 카운터
- `Gas` : 가스 한도
- `GasPrice` : 가스 가격 (Legacy, EIP-155)
- `CallerAddress` : 발신자 주소 (20바이트)
- `CalleeAddress` : 수신자 주소 (20바이트, 컨트랙트 생성 시 0)
- `IsCreate` : 컨트랙트 생성 여부 (1 또는 0)
- `Value` : 전송 값 (wei 단위)
- `ChainID` : 체인 ID (EIP-155)
- `BlockNumber` : 트랜잭션이 포함된 블록 번호

CallData 필드

- `CallData` : 개별 바이트 (index로 접근)
- `CallDataLength` : CallData 총 길이
- `CallDataRLC` : CallData의 RLC 인코딩
- `CallDataGasCost` : CallData 가스 비용 (0바이트: 4 gas, 비0바이트: 16 gas)
- `TxDataGasCost` : L1에 청구되는 데이터 가스 비용

서명 필드

- `SigV` : ECDSA 서명 V (recovery ID)

- `SigR` : ECDSA 서명 R (32바이트)
- `SigS` : ECDSA 서명 S (32바이트)
- `TxSignLength` : 서명 전 RLP 인코딩 길이
- `TxSignRLC` : 서명 전 RLP 인코딩 RLC
- `TxSignHash` : 서명할 메시지 해시
- `TxHashLength` : 서명 후 RLP 인코딩 길이
- `TxHashRLC` : 서명 후 RLP 인코딩 RLC
- `TxHash` : 최종 트랜잭션 해시

EIP-2930 Access List 필드

- `AccessListAddress` : Access List의 주소
- `AccessListStorageKey` : Access List의 스토리지 키
- `AccessListAddressesLen` : Access List 주소 개수
- `AccessListStorageKeysLen` : Access List 스토리지 키 총 개수
- `AccessListRLC` : Access List의 RLC 인코딩

EIP-1559 필드

- `MaxPriorityFeePerGas` : 최대 우선순위 수수료
- `MaxFeePerGas` : 최대 수수료

트랜잭션 타입

- `TxType` : 트랜잭션 타입
- `0` : Legacy (pre-EIP-155)
- `1` : EIP-2930 (Access List)
- `2` : EIP-1559 (Dynamic Fee)

- `0x7E` : L1 Message (Scroll 확장)

테이블 레이아웃

Tx Table은 **고정 섹션**과 **동적 섹션**으로 나뉩니다.

고정 섹션 (Fixed Section)

각 트랜잭션에 대해 **고정된 수의 행**을 할당합니다. CallData와 Access List를 제외한 모든 필드가 포함됩니다.

트랜잭션당 행 수: 약 25-30행 (트랜잭션 타입에 따라 다름)

예시 (tx_id = 1):

```
tx_id | tag | index | value | access_list_address

------|------------------|-------|-----------------|--------------------

1 | Nonce | 0 | 5 | 0

1 | GasPrice | 0 | 20000000000 | 0

1 | Gas | 0 | 21000 | 0

1 | CallerAddress | 0 | 0x1234... | 0

1 | CalleeAddress | 0 | 0x5678... | 0

1 | IsCreate | 0 | 0 | 0

1 | Value | 0 | 1000000000000 | 0

1 | CallDataLength | 0 | 100 | 0

1 | CallDataRLC | 0 | rlc_value | 0

1 | CallDataGasCost | 0 | 1600 | 0

1 | ChainID | 0 | 1 | 0
```

```
1 | SigV | 0 | 27 | 0

1 | SigR | 0 | r_value | 0

1 | SigS | 0 | s_value | 0

1 | TxSignLength | 0 | 85 | 0

1 | TxSignRLC | 0 | rlc_value | 0

1 | TxSignHash | 0 | hash_value | 0

1 | TxHashLength | 0 | 150 | 0

1 | TxHashRLC | 0 | rlc_value | 0

1 | TxHash | 0 | hash_value | 0

1 | TxType | 0 | 0 | 0

1 | BlockNumber | 0 | 12345 | 0

...
```

동적 섹션 (Dynamic Section)

CallData와 Access List는 **가변 길이**이므로 동적 섹션에 저장됩니다.

CallData 행

각 CallData 바이트에 대해 하나의 행:

```
tx_id | tag | index | value | access_list_address

------|----------|-------|-------|--------------------

1 | CallData | 0 | 0x60 | 0

1 | CallData | 1 | 0x80 | 0
```

```
1 | CallData | 2 | 0x60 | 0

...

1 | CallData | 99 | 0x00 | 0
```

Access List 행

주소 행:

```
tx_id | tag | index | value | access_list_address

------|--------------------| ------|-------|--------------------

1 | AccessListAddress | 0 | 0 | 0xABCD...

1 | AccessListAddress | 1 | 0 | 0xEF01...
```

스토리지 키 행:

```
tx_id | tag | index | value (storage_key) | access_list_address

------|------------------------|-------|---------------------|------------------

1 | AccessListStorageKey | 0 | key1_rlc | 0xABCD...

1 | AccessListStorageKey | 1 | key2_rlc | 0xABCD...

1 | AccessListStorageKey | 2 | key3_rlc | 0xEF01...
```

패딩

블록에 트랜잭션이 `max_txs` 보다 적은 경우, **더미 트랜잭션**으로 패딩됩니다:

```
let  padding_txs = (txs.len()..max_txs)
```

```
.map(|tx_id| {
let mut padding_tx = Transaction::dummy(chain_id);
padding_tx.id = tx_id + 1;
padding_tx
})
.collect::<Vec<Transaction>>();
```

더미 트랜잭션은 유효한 트랜잭션 구조를 가지지만 실제로 실행되지 않습니다.

트랜잭션 타입별 필드

Legacy 트랜잭션 (Type 0, pre-EIP-155)

필수 필드:

- `Nonce`, `GasPrice`, `Gas`, `CallerAddress`, `CalleeAddress`, `IsCreate`, `Value`
- `CallData`, `CallDataLength`, `CallDataRLC`, `CallDataGasCost`
- `SigV`, `SigR`, `SigS`
- `TxSignLength`, `TxSignRLC`, `TxSignHash`
- `TxHashLength`, `TxHashRLC`, `TxHash`
- `TxType` (= 0)

특징:

- `ChainID` 없음 (replay attack 취약)
- `SigV`: 27 또는 28

EIP-155 트랜잭션 (Type 0)

추가 필드:

- `ChainID` : 체인 ID

특징:

- `SigV` : 35 + 2 * chain_id 또는 36 + 2 * chain_id
- Replay attack 방지

EIP-2930 트랜잭션 (Type 1)

추가 필드:

- `AccessListAddress` , `AccessListStorageKey`
- `AccessListAddressesLen` , `AccessListStorageKeysLen`
- `AccessListRLC`

특징:

- Access List를 통해 가스 비용 최적화
- `SigV` : 0 또는 1 (recovery ID)

EIP-1559 트랜잭션 (Type 2)

추가 필드:

- `MaxPriorityFeePerGas` : 최대 우선순위 수수료
- `MaxFeePerGas` : 최대 수수료

변경 필드:

- `GasPrice` 대신 동적 수수료 사용

특징:

- Base fee + Priority fee 모델
- 가스 가격 예측 가능성 향상

L1 Message 트랜잭션 (Type 0x7E, Scroll 확장)

특징:

- L1에서 L2로 전송되는 메시지
- 서명 없음 (`SigV` , `SigR` , `SigS` = 0)
- L1 컨트랙트에서 검증됨

EVM Circuit과의 연동

EVM Circuit은 Tx Table을 조회하여 트랜잭션 정보를 가져옵니다.

트랜잭션 필드 Lookup

```
// EVM Circuit에서 Tx Table lookup
meta.lookup_any("tx field lookup", |meta| {
vec![
1.expr(), // q_enable
tx_id.expr(), // tx_id
TxFieldTag::CallerAddress.expr(), // tag
0.expr(), // index
caller_address.expr(), // value
0.expr(), // access_list_address
]
});
```

CallData Lookup

```
// CallData 바이트 조회
meta.lookup_any("calldata lookup", |meta| {
vec![
1.expr(), // q_enable
tx_id.expr(), // tx_id
TxFieldTag::CallData.expr(), // tag
calldata_index.expr(), // index
calldata_byte.expr(), // value
0.expr(), // access_list_address
]
});
```

사용 예시

CALLER opcode:

```
// 현재 트랜잭션의 발신자 주소 조회
let  caller = lookup_tx_field(tx_id, TxFieldTag::CallerAddress);
stack.push(caller);
```

CALLDATALOAD opcode:

```
// CallData에서 32바이트 읽기
```

```
let offset = stack.pop();
let mut data = 0;
for i in 0..32 {
let byte = lookup_tx_field(tx_id, TxFieldTag::CallData, offset + i);
data = (data << 8) | byte;
}
stack.push(data);
```

Tx Circuit과의 연동

Tx Circuit은 Tx Table에 데이터를 할당하고 검증합니다.

데이터 할당

파일: `zkevm-circuits/src/table.rs` (line 263)

```
impl TxTable {
pub fn load<F: Field>(
&self,
layouter: &mut impl Layouter<F>,
txs: &[Transaction],
max_txs: usize,
max_calldata: usize,
chain_id: u64,
challenges: &Challenges<Value<F>>,
```

```
) -> Result<Vec<AssignedCell<F, F>>, Error> {

// 1. All-zero 행 할당

// 2. 고정 섹션 할당 (각 트랜잭션의 기본 필드)

// 3. 동적 섹션 할당 (CallData, Access List)

// 4. 청크 트랜잭션 바이트 해시 할당

}

}
```

서명 검증

Tx Circuit은 Sig Table을 조회하여 서명을 검증합니다:

```
// Sig Table lookup

meta.lookup_any("tx signature", |meta| {

vec![

1.expr(), // q_enable

tx_sign_hash.expr(), // msg_hash

sig_v.expr(), // sig_v

sig_r.expr(), // sig_r

sig_s.expr(), // sig_s

caller_address.expr(), // recovered_addr

1.expr(), // is_valid

]

});
```

RLP 인코딩 검증

Tx Circuit은 RLP Table을 조회하여 RLP 인코딩을 검증합니다:

```
// RLP Table lookup
meta.lookup_any("tx rlp", |meta| {
vec![
tx_id.expr(), // tx_id
rlp_format.expr(), // format
rlp_tag.expr(), // rlp_tag
tag_value.expr(), // tag_value
tag_bytes_rlc.expr(), // tag_bytes_rlc
tag_length.expr(), // tag_length
]
});
```

청크 트랜잭션 바이트 해시

멀티 블록 증명에서는 **청크 트랜잭션 바이트 해시**가 사용됩니다.

계산 방식

```
// 청크 내 모든 L2 트랜잭션의 RLP 바이트 수집
let  chunk_txbytes = txs
.iter()
```

```
.flat_map(|tx| {
if  tx.is_chunk_l2_tx() {
tx.rlp_signed.clone()
} else {
vec![]
}
})
.collect::<Vec<u8>>();


// Keccak256 해시 계산
let  chunk_txbytes_hash = keccak256(chunk_txbytes);


// RLC 인코딩
let  chunk_txbytes_hash_rlc = rlc_be_bytes(
&chunk_txbytes_hash.to_fixed_bytes(),
challenges.evm_word()
);
```

할당 위치

`chunk_txbytes_hash_rlc` 는 고정 섹션의 **마지막 행**에 할당됩니다:

```
// 고정 섹션 마지막 행에 할당
region.assign_advice(
```

```
|| "tx table chunk txbytes hash rlc",
self.chunk_txbytes_hash_rlc,
offset - 1, // 마지막 행
|| chunk_txbytes_hash_rlc,
)?;
```

사용 목적

- 청크 간 트랜잭션 데이터 무결성 검증
- Public Input Circuit에서 사용
- 증명 집계 시 청크 연결

성능 최적화

1. 고정/동적 섹션 분리

- **고정 섹션**: 모든 트랜잭션에 대해 동일한 크기
- **동적 섹션**: CallData와 Access List만 포함
- 이를 통해 Fixed 컬럼 태그를 일관되게 유지

2. SecondPhase 컬럼

RLC를 사용하는 컬럼들은 SecondPhase에 할당:

- `value` : RLC 값 포함
- `chunk_txbytes_hash_rlc` : 청크 해시 RLC

3. 패딩 최적화

더미 트랜잭션은 최소한의 CallData만 포함하여 테이블 크기 최소화

제약 조건

1. 서명 검증

```
// Sig Table을 통한 서명 검증
// recovered_address == caller_address
```

2. RLP 인코딩 검증

```
// RLP Table을 통한 인코딩 검증
// keccak(rlp_encoded) == tx_hash
```

3. CallData 가스 비용

```
// CallData 가스 비용 계산
// gas_cost = Σ (byte == 0 ? 4 : 16)
```

4. Access List 일관성

```
// Access List 주소와 스토리지 키 개수 일치
// addresses_len == 주소 행 개수
// storage_keys_len == 스토리지 키 행 개수
```

예시: 전체 트랜잭션

트랜잭션:

```
From: 0x1234...

To: 0x5678...

Value: 1 ETH

Gas: 21000

GasPrice: 20 Gwei

Nonce: 5

CallData: 0x60806040...
```

Tx Table (일부):

```
tx_id | tag | index | value

------|------------------|-------|------------------

1 | Nonce | 0 | 5

1 | GasPrice | 0 | 20000000000

1 | Gas | 0 | 21000

1 | CallerAddress | 0 | 0x1234... (RLC)

1 | CalleeAddress | 0 | 0x5678... (RLC)

1 | Value | 0 | 1000000000000000000

1 | CallDataLength | 0 | 4

1 | CallData | 0 | 0x60

1 | CallData | 1 | 0x80
```

```
1 | CallData | 2 | 0x60

1 | CallData | 3 | 0x40
```

디버깅 팁

트랜잭션 덤프

```
// 트랜잭션 정보 출력
for tx in txs.iter() {
println!("Tx {}: from={:?}, to={:?}, value={}, gas={}",
tx.id, tx.caller_address, tx.callee_address, tx.value, tx.gas);
}
```

일반적인 오류

1. **서명 불일치**: 서명 검증 실패
2. **RLP 인코딩 오류**: 트랜잭션 해시 불일치
3. **CallData 가스 계산 오류**: 가스 비용이 맞지 않음
4. **Access List 불일치**: 주소/키 개수가 맞지 않음

관련 문서

- Tx Circuit - Tx Table을 생성하는 회로
- EVM Circuit - Tx Table을 조회하는 회로

- Sig Circuit - 서명 검증 회로
- RLC 기법 - RLC 인코딩 상세

참고 자료

- **코드**: `zkevm-circuits/src/table.rs` (line 212)
- **Witness**: `zkevm-circuits/src/witness/tx.rs`
- **EIP-155**: https://eips.ethereum.org/EIPS/eip-155
- **EIP-2930**: https://eips.ethereum.org/EIPS/eip-2930
- **EIP-1559**: https://eips.ethereum.org/EIPS/eip-1559